

Module 3: Data Organization and Manipulation using Collections

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai campus

premanand.s@vit.ac.in

September 9, 2025

In module 3,

- Data Handling Strategies Using Lists, Tuples, Dictionary and Set
- Data Comprehension
- Data Modification
- Grouping and Categorization
- Iterators
- Data Selection - Ordered, Unordered and Unique Data
- Searching Techniques: Linear Search, Binary Search
- Sorting Techniques: Bubble Sort, Selection Sort- Insertion Sort- Quick Sort- Merge Sort.

What is Searching?

- Searching means finding something you want from a group.
- searching = looking for something in a collection.
- Example:
 - You have a toy box with 10 toys.
 - If you want to find your red car, you will search inside the box until you find it.

Types - Searching?

- Linear Search (basic)
- Binary Search (basic-smart)
- Jump Search (medium-smart)
- Interpolation Search (very smart when data is uniform)
- Exponential Search
- Hashing (ultra-fast direct search)
- Fibonacci Search
- Ternary Search
- Search in Trees / Graphs (Advanced) - DFS, BFS
- Search in Databases

Types - Searching?

- Linear Search → Checking one by one.
- Binary Search → Smart searching, jumping to the middle.

Linear Search

- book - 'Python for Everybody'
- Starts checking - comparing - stack of books
- till you FOUND
- Linear Search → check one by one until you find it.

Binary Search (Smart Middle Jump)

- Now imagine you have numbers already arranged in order: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
- You want to find 7.
- Step 1: Check the middle (5). Is $7 = 5$? Is 7 bigger than 5? Yes $\rightarrow$ so ignore the left side.
- Step 2: New list is [6, 7, 8, 9, 10]. Middle is 8. Is $7 = 8$? Is 7 smaller than 8? Yes $\rightarrow$ so ignore the right side.
- Step 3: New list is [6, 7]. Middle is 6. Is $7 = 6$? Is 7 bigger? Yes $\rightarrow$ go right.
- Step 4: Left is only [7] $\rightarrow$ Found
- Binary Search $\rightarrow$ jump to the middle each time, cutting the list in half.

Quick Analogy

- Linear Search → Like checking each page in your notebook one by one to find a picture.
- Binary Search → Like opening the middle page first, then deciding whether the picture is before or after.

When to Use?

- If things are not in order → Linear search (toys scattered).
- If things are in order → Binary search (like a phone book or dictionary).

Linear Search Illustration

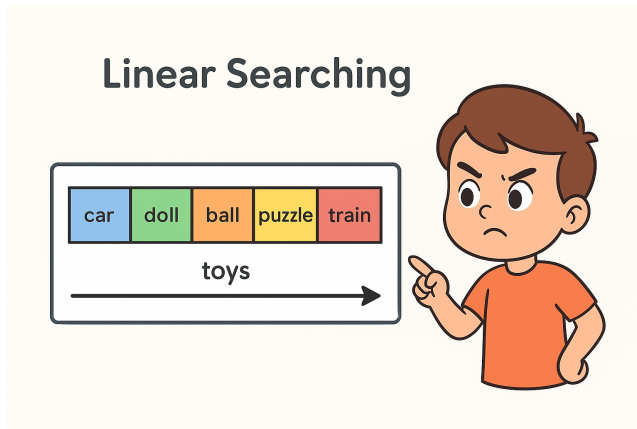


Figure: Linear Search process: repeatedly starts from first till FIND.

Example (Syntax)

```
toys = ["car", "doll", "ball", "puzzle", "train"]

search_item = "ball"

for toy in toys:
    if toy == search_item:
        print("Yes! Found the toy:", toy)
        break
```

Example (Syntax)

```
candies = ["lollipop", "chocolate", "gummy", "mint"]
search_item = "gummy"

for candy in candies:
    if candy == search_item:
        print("Yay! Got my candy:", candy)
        break
```

Linear Searching - not found

Example (Syntax)

```
toys = ["car", "doll", "ball", "puzzle", "train"]
search_item = "robot"

found = False
for toy in toys:
    if toy == search_item:
        print("Found:", toy)
        found = True
        break

if not found:
    print("Sorry, not in the toy box!")
```

Linear Searching - Finding the Position

Example (Syntax)

```
toys = ["car", "doll", "ball", "puzzle", "train"]
search_item = "puzzle"

for i in range(len(toys)):
    if toys[i] == search_item:
        print("Found at position:", i)
        break
```

Linear Searching - Multiple Occurrences

Example (Syntax)

```
candies = ["mint", "choco", "mint", "lolly", "mint"]
search_item = "mint"

for i in range(len(candies)):
    if candies[i] == search_item:
        print("Found", search_item, "at position", i)
```

Complexity

- If you have 5 toys, you may need to check all 5.
- If you have 100 toys, you may need to check all 100.
- So more toys $\rightarrow$ takes more time.
- $O(n)$ (Big-O Notation)
- The number of steps is almost the same as the number of toys (items) in the box.

Example (Syntax)

```
toys = ["car", "doll", "ball", "puzzle", "train"]
```

Want car? → check 1 step

Want train? → check 5 steps

Want robot (not there)? → check all 5 steps

That's why linear search is simple but slow if you have too many items.

Questions - Linear Search

- Search for the number 7 in the list [1, 3, 5, 7, 9]
- Check if "apple" exists in the list ["banana", "mango", "apple", "grapes"].
- Find whether the number 10 is present in [2, 4, 6, 8].
- Find the position of 25 in the list [10, 20, 25, 30, 35].
- If a number occurs multiple times (e.g., search 3 in [3, 5, 3, 7, 3]), return all the positions where it appears.
- Search for "dog" in the list ["cat", "dog", "rabbit", "dog", "parrot"] and count how many times "dog" appears.

Questions - Linear Search

- Write a program to search for a number in a list. If found → print "Yes, found!", else "Not found".
- Modify your linear search to return -1 if not found, otherwise return the index.
- Search for the first vowel (a, e, i, o, u) in a word given as a string (treat string as a list of characters).
- Your school has roll numbers [101, 102, 103, 104]. Write a program to check if roll number 103 is present.
- A shop has items ["pen", "pencil", "eraser", "scale"]. Search for the item "marker". If not found, print "Sorry, not available".
- A teacher wants to check if a student "Krithiksha" is in the list of names ["Anu", "Santhu", "Nikhil", "Krithiksha"].

Questions - Linear Search

- Given a list of numbers, write a linear search that finds the largest element.
- Modify linear search to return the second occurrence of an element. Example: search 5 in [2, 5, 7, 5, 9] → should return index 3.
- Write a function using linear search that searches multiple items at once (e.g., search [3, 7] in [1, 3, 5, 7, 9]).
- Create a linear search program that ignores case when searching in a list of strings (e.g., "Apple" should match "apple").
- Use linear search to find the minimum number of comparisons needed if the element is found at index i .

Binary Search Illustration

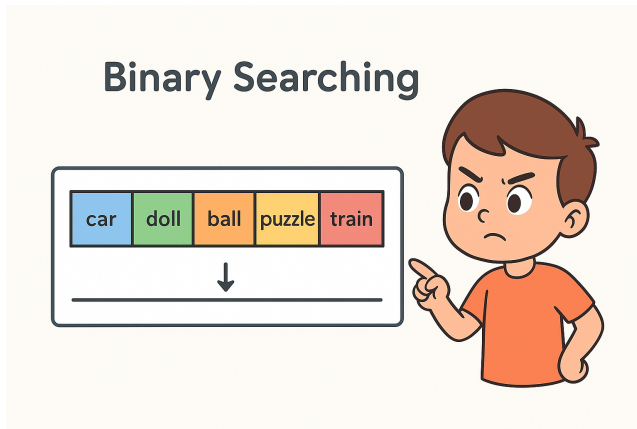


Figure: Binary Search process: repeatedly dividing the list in half.

Binary Search

Example (Syntax)

```
numbers = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72] # Sorted  
target = 23
```

```
low = 0
```

```
high = len(numbers) - 1
```

```
found = False
```

Binary Search

Example (Syntax)

```
while low <= high:
    mid = (low + high) // 2
    if numbers[mid] == target:
        print("Found at index:", mid)
        found = True
        break
    elif numbers[mid] < target:
        low = mid + 1
    else:
        high = mid - 1

if not found:
    print("Not found")
```

Quick Analogy

- Imagine you have a dictionary (book with words in order).
- You want to find the word “Tiger”.
 - You don't start from page 1.
 - Instead, you open the book in the middle. (normally)
 - If the word is before that page → look in the first half.
 - If the word is after that page → look in the second half.
 - Keep cutting the book into halves until you find the word.

How It Works

- Start with a sorted list
- Check the middle element.
- If it's what you want - FOUND!
- If your number is smaller → search in the left half.
- If your number is bigger → search in the right half.
- Repeat until found (or list is empty)

- Linear search checked every toy one by one.
- Binary search cuts the toy box into halves every time.
 - 8 toys $\rightarrow$ need max 3 tries.
 - 16 toys $\rightarrow$ need max 4 tries.
 - 32 toys $\rightarrow$ need max 5 tries.
- $O(\log n)$ (time grows much slower).

Example

- 101, 105, 110, 120, 125, 130, 135, 140
- $low = 0, high = 7$
- $mid = (low + high) // 2 = (0 + 7) // 2 = 3$
- $numbers[mid] = numbers[3] = 120$
- Compare with target 120 $\rightarrow$ equal - FOUND!

Example

- 101, 105, 110, 120, 125, 130, 135, 140
- $\text{low} = 0$, $\text{high} = 7$, $\text{mid} = (0 + 7) // 2 = 3$, $\text{numbers}[3] = 120$,
Compare: 120 ; 135 $\rightarrow$ so search right half, Update: $\text{low} = \text{mid} + 1 = 4$, $\text{high} = 7$.
- $\text{low} = 4$, $\text{high} = 7$, $\text{mid} = 5$, $\text{numbers}[5] = 130$, Compare: 130 ; 135
 $\rightarrow$ still too small $\rightarrow$ search right half, Update: $\text{low} = \text{mid} + 1 = 6$,
 $\text{high} = 7$.
- $\text{low} = 6$, $\text{high} = 7$, $\text{mid} = 6$, $\text{numbers}[6] = 135$, Compare: $135 == 135$, Found at index 6

Questions,

- Search for the number 7 in [1, 3, 5, 7, 9, 11, 13].
- Search for 20 in [10, 20, 30, 40, 50].
- Search for 25 in [5, 10, 15, 20, 30, 35].
- Find the index of 90 in [10, 20, 30, 40, 50, 60, 70, 80, 90].
- Try to find 100 in [10, 20, 30, 40, 50]. What should the program return if it's not found?
- Given [2, 4, 6, 8, 10, 12], search for 3. Show the steps of middle checks.

Questions,

- Write a program using binary search to return index of element if found, else -1.
- Modify binary search to count the number of comparisons made while searching.
- For the sorted list [1, 2, 3, 4, 5, 6, 7, 8], search for 6 and explain which halves get eliminated step by step.
- A teacher has roll numbers [101, 105, 110, 120, 125, 130, 135, 140]. Use binary search to check if roll number 120 exists.
- A shop keeps prices [50, 100, 150, 200, 250, 300]. Write a program to search for a price entered by a customer.
- A dictionary app stores words in order. Write a binary search function to check if "apple" is in the sorted word list ["ant", "apple", "bat", "cat", "dog"].

Questions,

- Given $[2, 4, 4, 4, 6, 8]$, write a binary search that returns the first occurrence of 4.
- Modify binary search to return the last occurrence of a number.
- Write a binary search function that works on a list of strings (e.g., names).
- Use binary search to find the smallest element greater than target (upper bound). Example: list = $[10, 20, 30, 40]$, target = 25 $\rightarrow$ output = 30.
- Given a sorted list of numbers from 1 to 1,000, use binary search to find how many steps it takes to find 999.
- Compare Linear Search vs Binary Search for the list $[1, 2, 3, \dots, 1000]$. How many steps maximum in each case?

- Python library that gives you:
 - Ready-made implementations of algorithms.
 - Step-by-step explanation of how they work.
 - Even the time complexity.

Example (Syntax)

```
pip install pygorithm
```

Example (Syntax)

```
import pygorithm  
help(pygorithm)
```

Example (Syntax)

```
import pygorithm  
print(dir(pygorithm))
```

Example (Syntax)

```
import pygorithm.searching  
print(dir(pygorithm.searching))
```

Get Functions Inside Each Algorithm

Example (Syntax)

```
from pygorithm.searching import binary_search  
print(dir(binary_search))
```

Linear Search with Pygorithm

Example (Syntax)

```
from pygorithm.searching import linear_search

numbers = [10, 20, 30, 40, 50]
index = linear_search.search(numbers, 30)

print("Index of 30:", index)

print(linear_search.get_code())

print(linear_search.time_complexities())
```

Binary Search with Pygorithm

Example (Syntax)

```
from pygorithm.searching import binary_search

numbers = [10, 20, 30, 40, 50]

index = binary_search.search(numbers, 40)
print("Index of 40:", index)

print(binary_search.get_code())

print(binary_search.time_complexities())
```

Binary Search with Pygorithm

Example (Syntax)

```
import time
from pygorithm.searching import linear_search, binary_search

nums = list(range(1, 1000000))

start = time.time()
linear_search.search(nums, 999999)
end = time.time()
print("Linear Search Time:", end - start)

start = time.time()
binary_search.search(nums, 999999)
end = time.time()
print("Binary Search Time:", end - start)
```